

Name of the Student	Ch. Venkata Lakshmi Hymavathi
Reg. No	192425397
GitHub Link	

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning
SLOT : D

COURSE CODE: MLA0405
Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I **Ch. Venkata Lakshmi Hymavathi (192425397)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Ch. Hymavathi

Name of the student:

Ch. Venkata Lakshmi Hymavathi

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature :

Total Marks:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

Answer: **Problem**

A manufacturing company needs to detect defective products using **high-dimensional sensor data**. Since the sensor data contains many features, an **Undercomplete Autoencoder (UAE)** can learn a compact representation and remove irrelevant information.

Proposed Architecture



Design

- **Input:** 100 sensor features.
- **Encoder:** $100 \rightarrow 64 \rightarrow 32 \rightarrow 10$.
- **Latent space:** 10 neurons.
- **Decoder:** $10 \rightarrow 32 \rightarrow 64 \rightarrow 100$.
- **Hidden activation:** ReLU.
- **Output activation:** Linear for continuous sensor values.
- **Loss function:** Mean Squared Error (MSE).

Reconstruction Loss

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

where:

- x_i = original sensor value
- $\hat{x}_i$ = reconstructed sensor value.

Working

The encoder compresses the high-dimensional sensor data into a **10-dimensional latent representation**. The decoder reconstructs the original input from this representation. During training, the network minimizes reconstruction error.

After training, the latent features can be given to a classifier such as a **Random Forest, SVM, or neural network** to classify products as defective or non-defective.

Advantages

1. Reduces dimensionality from 100 to 10 features.
2. Removes redundant information.
3. Learns useful features automatically.

Expected outcome: The autoencoder learns compact and meaningful sensor representations that improve defect detection.

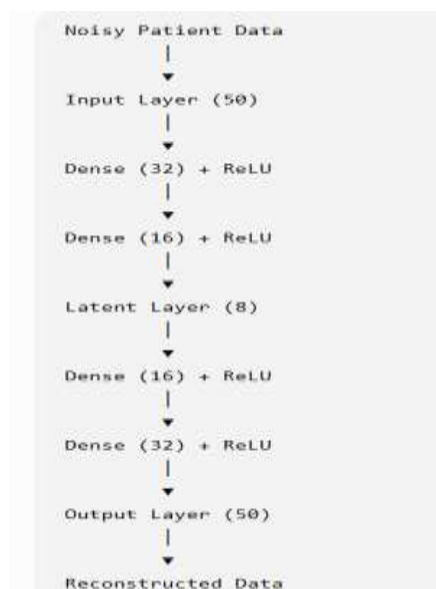
2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

Answer: **Problem**

Healthcare records may contain **noise, missing values, and irrelevant variations**. A regularized autoencoder can learn stable representations while reducing overfitting. The question specifically asks for a regularized design and explanation of its training objective.

Proposed Architecture



Regularization Strategy

Use **L2 regularization** on the encoder weights.

The total objective is:

$$L = L_{reconstruction} + \lambda \sum W^2$$

where:

- $L_{reconstruction}$ = reconstruction error
- W = network weights
- λ = regularization coefficient.

For example:

$$L = MSE(x, \hat{x}) + \lambda ||W||_2^2$$

Working

1. Noisy patient records are given as input.
2. Encoder compresses the data.
3. Latent layer learns important patient features.
4. Decoder reconstructs the original data.
5. Reconstruction error is calculated.
6. L2 penalty prevents excessively large weights.
7. The model learns general features rather than memorizing training data.

Why Regularization Is Needed

Without regularization, the model may simply memorize the training records. L2 regularization constrains the model and therefore helps reduce **overfitting**.

Advantages

- Robust to noisy data.
- Reduces overfitting.
- Learns meaningful representations.

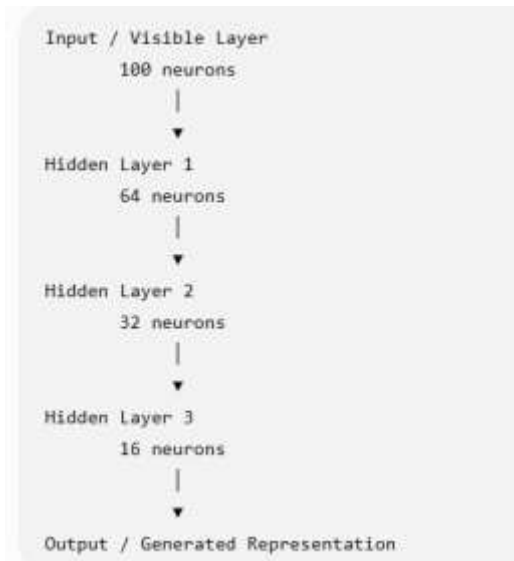
Expected outcome: The regularized autoencoder produces stable and useful patient representations that improve classification performance.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

Answer: For a recommendation/content-generation system, a **Deep Belief Network (DBN)** can learn hierarchical representations of complex data. The assessment asks for a DBN or DBM with visible and hidden layers and its learning process.

Proposed DBN Architecture



Design

- **Visible layer:** Represents user preferences/content features.
- **Hidden layer 1:** 64 neurons.
- **Hidden layer 2:** 32 neurons.
- **Hidden layer 3:** 16 neurons.
- **Activation:** Sigmoid for binary-valued features.
- The DBN is constructed using multiple **Restricted Boltzmann Machines (RBMs)**.

Learning Process

The DBN is trained in two major stages.

Step 1: Layer-wise pretraining

Each RBM is trained individually using unsupervised learning.

Visible → Hidden 1

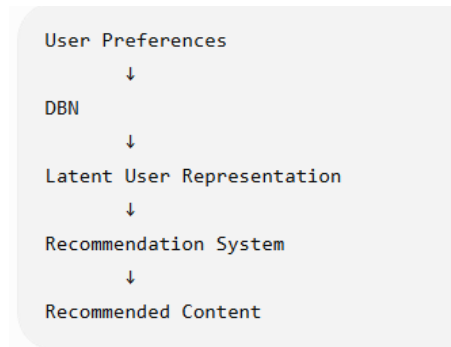
Hidden 1 → Hidden 2

Hidden 2 → Hidden 3

Step 2: Fine-tuning

After pretraining, the complete network is fine-tuned for the recommendation or content-generation task.

Application



Why DBN?

1. Learns hierarchical features.
2. Handles complex distributions.
3. Uses unsupervised pretraining.
4. Learns useful latent representations.
5. Suitable for recommendation and content-generation applications.

Evaluation

- Recommendation accuracy
- Precision@K
- Recall@K
- RMSE for rating prediction
- Quality/diversity of generated content

Expected outcome: The DBN learns complex hidden patterns in user preferences and generates useful representations for personalized recommendations.

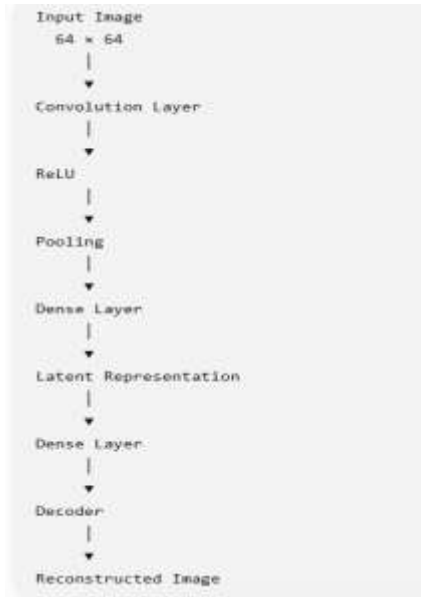
4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

Answer:

An image or signal-processing system needs representations that remain stable even when the input has small changes. A **Contractive Autoencoder (CAE)** is suitable because it encourages the encoder to become insensitive to small input variations. The assessment specifically requires the architecture, objective, and robustness mechanism.

Proposed Architecture



Contractive Objective

The loss contains two components:

$$L = L_{reconstruction} + \lambda L_{contractive}$$

where:

$$L_{reconstruction} = ||x - \hat{x}||^2$$

and the contractive penalty is based on the encoder's Jacobian:

$$L_{contractive} = ||J_f(x)||_F^2$$

Therefore:

$$L = ||x - \hat{x}||^2 + \lambda ||J_f(x)||_F^2$$

Working

1. Input image/signal is given to the encoder.
2. Encoder produces a latent representation.
3. Decoder reconstructs the input.
4. Reconstruction error measures how accurately the input is recovered.
5. Contractive penalty forces the representation to change slowly when the input changes slightly.
6. The model therefore learns stable features.

Robustness Mechanism

Suppose an image contains small changes such as:

- Slight brightness change
- Small noise
- Minor movement
- Small distortion

The latent representation should remain approximately similar.

Advantages

- Stable feature representation.
- Robust to small input variations.
- Reduces sensitivity to noise.
- Useful for image and signal processing.
- Learns locally invariant features.

Evaluation

- Reconstruction error
- Robustness under noise
- Feature stability
- Classification accuracy

Expected outcome: The model produces representations that remain useful even when the input undergoes small variations.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

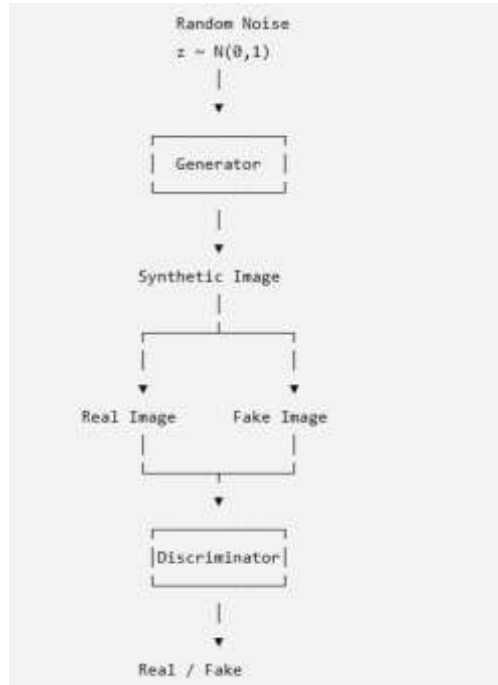
Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Answer:

An organization has limited training images and wants to generate realistic synthetic images. A **Generative Adversarial Network (GAN)** consists of a Generator and Discriminator trained against each other.

GAN Architecture



Generator

The Generator takes random noise as input and produces a synthetic image.

Discriminator

The Discriminator receives either a real image or generated image.

Image (64×64×3)

↓
Convolution
↓
ReLU
↓
Pooling
↓
Dense
↓
Sigmoid
↓
Real / Fake

Adversarial Training

The two networks compete with each other.

Generator:

Attempts to generate realistic images.

Tries to fool the discriminator.

Discriminator:

Attempts to identify real and fake images correctly.

Training process:

1. Generate random noise

↓

2. Generator creates fake image

- ↓
3. Give real + fake images to discriminator
- ↓
4. Calculate discriminator loss
- ↓
5. Calculate generator loss
- ↓
6. Update both networks
- ↓
7. Repeat until generated images become realistic

Loss Function

The traditional GAN objective is:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

The discriminator tries to correctly classify real and fake images, while the generator tries to make fake images appear real.

Evaluation

Generated images can be evaluated using:

Visual quality

Diversity

FID (Fréchet Inception Distance)

Inception Score

Downstream classification performance

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

